UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3069 - COMPUTACIÓN PARALELA Y DISTRIBUIDA - SECCIÓN - 10

Ing. Kimberly Barrera



Proyecto 02

Javier André Benítez García 23405

Javier Alejandro López del Cid 23415

Pedro Ruben Ávila Cofiño 23089

GUATEMALA, 21 de octubre de 2026

Índice

1. Introducción..... 3
2. Antecedentes conceptuales..... 3
2.1 AES-128 y su funcionamiento..... 3
2.2 Cifrado de texto y conversión de la clave..... 3
2.3 Búsqueda por fuerza bruta..... 4
2.4 Open MPI y métricas de rendimiento..... 4
3. Desarrollo..... 4
3.1 Programa base..... 4
3.2 Estrategia 1: reparto intercalado (base)..... 6
3.3 Estrategia 2: bloques contiguos (alternativa)..... 6
3.4 Terminación coordinada entre procesos..... 7
4. Resultados y análisis..... 8
4.1 Cifrado y descifrado..... 8
4.2 Búsqueda de la clave con 4 procesos..... 10
4.3 Por qué cambia el tiempo según la clave..... 10
4.4 Speedup..... 11
5. Discusión, conclusiones y recomendaciones..... 12
5.1 Retos y soluciones..... 12
5.2 Conclusiones..... 13
5.3 Recomendaciones..... 13
Anexo 1. Catálogo de funciones y librerías..... 14
Anexo 2. Bitácora de pruebas y speedups..... 18
Referencias..... 25

1. Introducción

En el Laboratorio 03 se analizó y mejoró un programa secuencial de búsqueda de claves AES (rango, clave y mensaje por parámetro, modo CBC con IV, reutilización del contexto de OpenSSL y validación de argumentos). Este proyecto parte de ese trabajo con tres objetivos: (1) cifrar y descifrar con AES-128 un texto leído desde un archivo `.txt`, usando una clave recibida como parámetro; (2) encontrar por fuerza bruta la clave usada, con 4 procesos MPI, midiendo el tiempo para tres claves de prueba; y (3) proponer, implementar y comparar una estrategia alternativa de distribución del rango de claves entre los procesos. Además, siguiendo el objetivo de optimizar el programa, se aceleró la prueba de cada candidata con las instrucciones AES-NI del procesador.

El informe presenta los antecedentes conceptuales, el diseño de las dos estrategias con sus diagramas y pseudocódigo, los resultados con su análisis y las conclusiones. Los anexos incluyen el catálogo de funciones y librerías y la bitácora de pruebas con los cálculos de Speedup.

2. Antecedentes conceptuales

2.1 AES-128 y su funcionamiento

AES (Advanced Encryption Standard) es un algoritmo de cifrado simétrico por bloques estandarizado por el NIST en 2001 [1]. Es una variante del algoritmo Rijndael, de Joan Daemen y Vincent Rijmen [2]. Usa la misma clave para cifrar y descifrar y procesa bloques de 128 bits (16 bytes). El tamaño de la clave determina el número de rondas:

Tabla 1. Variantes de AES.

Variante	Clave	Rondas	Subclaves
AES-128	128 bits (16 bytes)	10	11
AES-192	192 bits (24 bytes)	12	13
AES-256	256 bits (32 bytes)	14	15

La clave de 16 bytes se expande en 11 subclaves de 16 bytes. Para cifrar, cada bloque se carga en una matriz de estado de 4×4 bytes y se aplica `AddRoundKey` con la primera subclave; luego se repiten 9 rondas de `SubBytes`, `ShiftRows`, `MixColumns` y `AddRoundKey`, y una ronda final sin `MixColumns`. Para descifrar se usan las mismas subclaves en orden inverso y las transformaciones inversas (`InvShiftRows`, `InvSubBytes`, `InvMixColumns`). La clave solo interviene en `AddRoundKey`; con una clave distinta, el descifrado devuelve bytes sin sentido, y en eso se basa la búsqueda por fuerza bruta.

2.2 Cifrado de texto y conversión de la clave

Un texto de más de 16 bytes se divide en bloques y se cifra con un modo de operación [3]. ECB cifra cada bloque por separado, por lo que bloques iguales producen cifrados iguales. CBC combina cada bloque con el cifrado anterior mediante XOR y necesita un vector de inicialización (IV). Si el texto no completa el último bloque, se agrega relleno (PKCS#7). El programa del proyecto usa AES-128 en modo CBC con IV y relleno PKCS#7 mediante la API EVP de OpenSSL [4].

La clave llega como un número entero y debe convertirse a 128 bits. Se usa la representación de 16 bytes en orden big-endian, con ceros a la izquierda. Las tres claves de prueba del proyecto quedan así:

Tabla 2. Claves de prueba convertidas a 128 bits.

Clave (decimal)	Clave de 128 bits (hexadecimal)
123456	`0000000000000000000000000001E240`
18014398509481984 (2^56/4 = 2^54)	`00000000000000000040000000000000`
18014398509481985 (2^56/4 + 1)	`00000000000000000040000000000001`

2.3 Búsqueda por fuerza bruta

La búsqueda por fuerza bruta prueba claves candidatas hasta encontrar la correcta. Aquí se trata de un ataque con texto parcialmente conocido: para cada candidata se descifra el mensaje cifrado y se revisa si el resultado contiene la frase de búsqueda "es una prueba de". Si la contiene, se da por encontrada la clave; la probabilidad de que una clave incorrecta produzca por azar una frase exacta de 16 bytes es despreciable (del orden de 2^-128 por posición).

El espacio completo de AES-128 tiene 2^128 ≈ 3.4×10^38 claves y es inviable de recorrer. El proyecto explora el rango [0, 2^56), es decir, N = 72 057 594 037 927 936 candidatas. El texto de prueba, "Esta es una prueba de proyecto 2", mide 32 bytes; como el relleno PKCS#7 agrega un bloque completo cuando el texto ya es múltiplo de 16, el cifrado ocupa 48 bytes (tres bloques). La frase de búsqueda ocupa los bytes 5 a 20 (contando desde 0) y cruza el límite entre el primer y el segundo bloque, por lo que no basta la prueba de un solo bloque del Laboratorio 03. Para que la prueba funcione con cualquier texto y cualquier frase, el programa descifra el mensaje completo con cada candidata y busca la frase en cualquier posición del resultado.

2.4 Open MPI y métricas de rendimiento

MPI es el estándar de paso de mensajes para memoria distribuida [5] y Open MPI es una de sus implementaciones. Un programa se lanza con varios procesos (`mpirun -np 4`); cada proceso conoce su identificador (rank) y el total (size) dentro del comunicador `MPI_COMM_WORLD`. En este proyecto se usan `MPI_Bcast` para difundir el mensaje cifrado, `MPI_Isend`, `MPI_Iprobe` y `MPI_Recv` para el aviso de clave encontrada (`MPI_Iprobe` revisa sin bloquearse si hay un mensaje pendiente [6]), `MPI_Barrier` antes de medir, `MPI_Allreduce` y `MPI_Reduce` para reunir resultados, `MPI_Waitall` para cerrar los envíos y `MPI_Wtime` para medir el tiempo.

El rendimiento se mide con el Speedup, S(p) = T_secuencial / T_paralelo(p), y con la eficiencia, E(p) = S(p) / p [7].

3. Desarrollo

3.1 Programa base

El punto de partida es el programa secuencial mejorado del Laboratorio 03 (rango, clave y mensaje por línea de comandos, AES-128-CBC con IV, reutilización del contexto de OpenSSL por candidata y validación de argumentos). Para el proyecto se reorganizó en tres programas que comparten las rutinas de `src/common/aes_texto.c` (Anexo 1): `aes_texto` cifra (`-e`) o descifra (`-d`) un archivo con la clave como parámetro; `busqueda_secuencial` recorre las candidatas con un solo proceso y es la referencia para el

Speedup; y `busqueda_mpi` reparte la búsqueda entre procesos MPI con la estrategia 1 o 2 (`-e 1|2`). La Figura 1 muestra el flujo general.

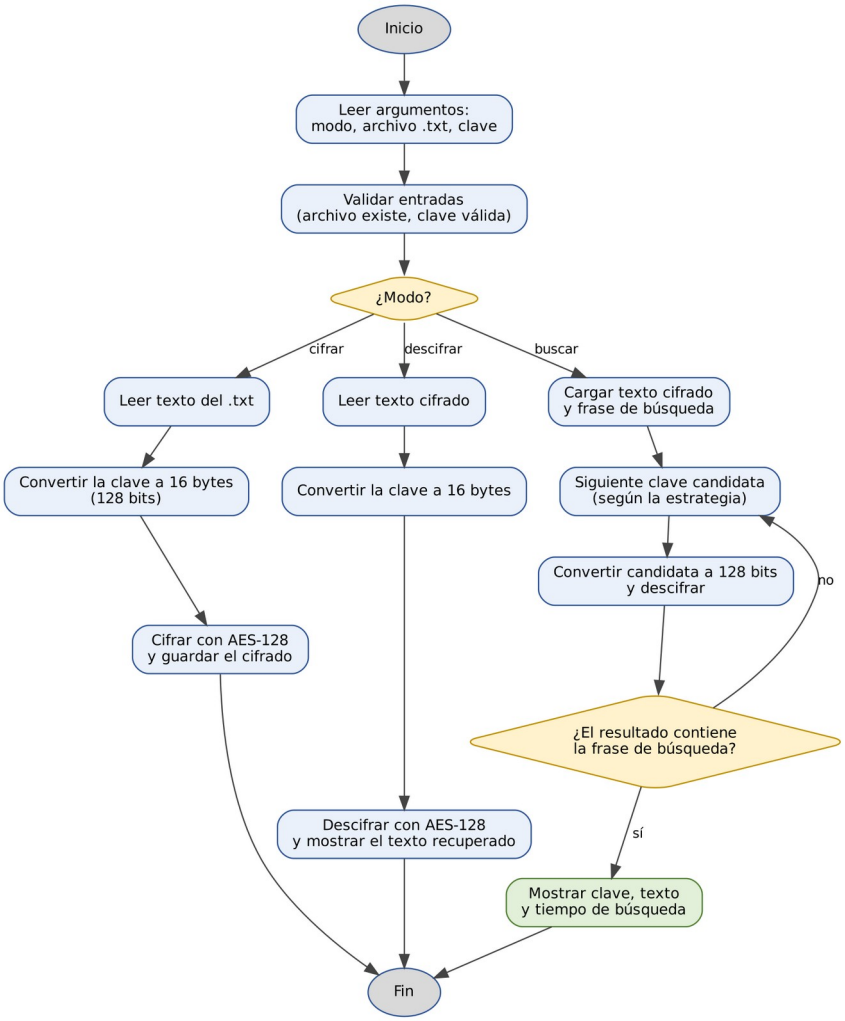


Figura 1. Flujo general: cifrar y descifrar (aes_texto) y buscar la clave (busqueda_secuencial y busqueda_mpi).

Conversión de la clave y prueba de una candidata. La clave numérica k se convierte con `clave_desde_u64()` en 16 bytes: los bytes 0 a 7 valen cero y los bytes 8 a 15 contienen k en big-endian (123456 → 0000000000000000000000000001E240). Cada candidata se prueba con `probar_clave()`: se descifran con AES-128-CBC y el IV del mensaje los 48 bytes del cifrado, sin validar todavía el relleno (con una clave incorrecta el último bloque es basura y no debe tratarse como error), y `contiene_frase()` busca "es una prueba de" en cualquier posición (`memchr` localiza el primer byte y `memcmp` compara el resto). Solo si la frase aparece se verifica el relleno PKCS#7, lo que descarta falsos positivos sin costo para las candidatas incorrectas. El contexto se prepara una sola vez por proceso en `buscador_iniciar()` y, por candidata, solo cambia la clave.

Optimización con AES-NI. En OpenSSL 3 cada `EVP_DecryptInit_ex` reinicia el contexto del proveedor, y para un mensaje de tres bloques ese costo fijo domina. Por eso se agregó una segunda implementación de la prueba que expande la clave y descifra CBC directamente con las instrucciones AES-NI del procesador (`_mm_aeskeygenassist_si128`, `_mm_aesimc_si128`, `_mm_aesdec_si128`). Se elige en tiempo de

ejecución (`-a auto|evp|aesni`): si la CPU no tiene AES-NI se usa EVP, y ambas rutas se verificaron contra OpenSSL. En el equipo de pruebas la tasa secuencial subió de 4.72×10^6 a 1.28×10^7 claves/s (2.7 veces, Tabla 7). Todas las mediciones del capítulo 4 usan AES-NI.

3.2 Estrategia 1: reparto intercalado (base)

Con p procesos, el proceso de rango r prueba las claves k = r, r + p, r + 2p, … menores que N (`calcular_tramo()` devuelve inicio = r y paso = p). Cada candidata pertenece a exactamente un proceso (el que cumple k mod p = r), así que ninguna se omite ni se repite. Todos los procesos avanzan a la vez sobre la misma zona del rango: la clave k se encuentra después de ⌊k/p⌋ + 1 intentos del proceso que la tiene, por lo que el tiempo es aproximadamente k / (p·v), donde v es la cantidad de claves por segundo de un proceso. Es la estrategia base: el Speedup es cercano a p sin importar dónde esté la clave, pero el tiempo crece con su posición.

```
ESTRATEGIA 1 (proceso r de p, rango [0, N))
  k <- r
  mientras k < N:
      si probar_clave(k): avisar_encontrada(k); terminar
      k <- k + p
      cada 4096 candidatas: si hay_aviso_found(): terminar
```

3.3 Estrategia 2: bloques contiguos (alternativa)

La estrategia alternativa divide el rango en p bloques contiguos y entrega uno a cada proceso. El proceso r recorre en orden ascendente el tramo [r·N/p, (r+1)·N/p); si N no es divisible entre p, los primeros N mod p procesos reciben una clave más. Con N = 2^56 y p = 4, cada bloque tiene 2^54 claves y los puntos de inicio son 0, 2^54, 2·2^54 y 3·2^54. La clave en el bloque j, con desplazamiento o dentro del bloque, se encuentra tras o + 1 intentos del proceso j, de modo que el tiempo es aproximadamente o / v. Las claves cercanas al inicio de cualquier bloque se hallan casi de inmediato, aunque estén muy lejos del inicio del rango. Las claves 2^56/4 y 2^56/4 + 1 son justamente la primera y la segunda del bloque del proceso 1.

```
ESTRATEGIA 2 (proceso r de p, rango [0, N))
  tam <- N div p ;  resto <- N mod p
  inicio <- r*tam + min(r, resto)
  fin    <- inicio + tam + (1 si r < resto, si no 0)
  para k desde inicio hasta fin - 1:
      si probar_clave(k): avisar_encontrada(k); terminar
      cada 4096 candidatas: si hay_aviso_found(): terminar
```

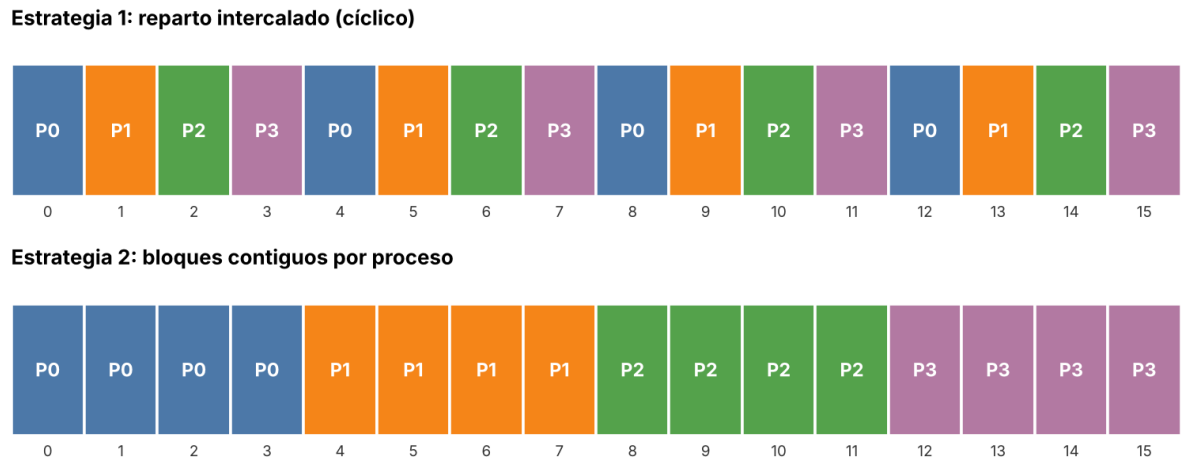


Figura 2. Reparto de las claves entre los procesos en cada estrategia.

Las dos estrategias reparten el mismo rango sin omitir ni repetir candidatas y hacen el mismo trabajo en el peor caso. Se diferencian en qué claves se visitan primero, y por eso el tiempo depende de la posición de la clave de forma distinta (ver 4.3).

3.4 Terminación coordinada entre procesos

Antes de buscar, el proceso 0 cifra el texto (o lee el archivo cifrado) y difunde el IV y el cifrado con `MPI_Bcast`, de modo que todos los procesos buscan exactamente el mismo mensaje. Los procesos se sincronizan con `MPI_Barrier` y cada uno toma el tiempo inicial con `MPI_Wtime`; así la medición no incluye el arranque de mpirun, la lectura del archivo ni la difusión.

Cuando un proceso encuentra la clave sale de su ciclo y la envía a cada uno de los demás con `MPI_Isend` (etiqueta FOUND, función `avisar_encontrada()`). El envío es no bloqueante: con `MPI_Send` el proceso podría quedarse esperando a un receptor que ya está en las reducciones finales, lo cual el estándar permite y se comprobó que bloquea el programa si se usa envío síncrono. Cada proceso revisa con `MPI_Iprobe` (`hay_aviso_found()`), cada 4096 candidatas, si llegó un aviso; si llegó, lo recibe con `MPI_Recv` y deja de buscar. En ese mismo punto se revisa el límite de tiempo de la opción `-t`. Revisar cada 4096 candidatas hace despreciable el costo de la comunicación, a cambio de que los demás procesos sigan probando al menos hasta su siguiente revisión después del hallazgo; en las corridas, con más procesos que núcleos, el aviso tardó a veces varios intervalos en llegar (el total de claves probadas del Anexo 2 lo muestra).

Al salir del ciclo (por hallazgo, aviso, límite de tiempo o rango agotado) todos participan en las reducciones: `MPI_Allreduce` con `MPI_MIN` obtiene la clave global (UINT64_MAX significa "no encontrada") y el proceso que la halló; con `MPI_Reduce` el proceso 0 reúne el tiempo del último proceso en detenerse (`MPI_MAX`) y el total de claves probadas (`MPI_SUM`). Antes de `MPI_Finalize`, `recibir_avisos_pendientes()` calcula con `MPI_Allreduce` cuántos procesos encontraron la clave, recibe exactamente los avisos que faltan; después, en `main`, el proceso que avisó espera sus envíos con `MPI_Waitall`, de modo que no queda ningún mensaje sin recibir. Si OpenSSL falla se llama a `MPI_Abort` para no dejar procesos esperando. El proceso 0 imprime la clave, el proceso que la halló, sus intentos y su tiempo, el texto recuperado, el tiempo total y la tasa; si la búsqueda se cortó con `-t`, imprime la estimación del tiempo para la clave de referencia (Figura 3).

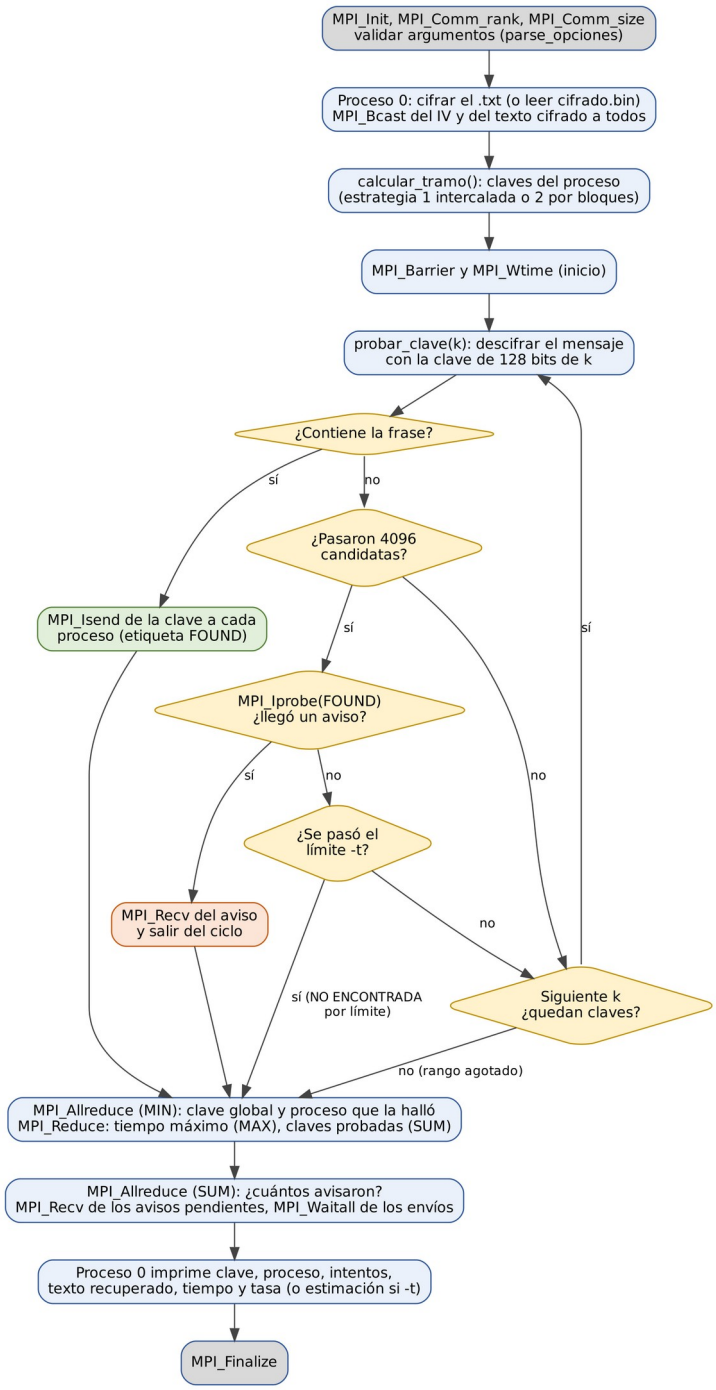


Figura 3. Flujo de cada proceso MPI: difusión del mensaje, búsqueda, aviso de la clave encontrada y cierre de la comunicación.

4. Resultados y análisis

4.1 Cifrado y descifrado

Las Figuras 4 y 5 muestran el cifrado y el descifrado del archivo `datos/prueba.txt` con la clave 123456, convertida a 128 bits (00000000000000000000000000001e240). Se ven el texto original, el IV aleatorio, el

texto cifrado de 48 bytes (32 bytes del texto más un bloque de relleno PKCS#7) y el texto recuperado, idéntico al original. La Figura 6 muestra que con otra clave (123457) el relleno no es válido y el programa lo rechaza. Todas las pruebas se ejecutaron en: Intel(R) Xeon(R) Processor @ 2.10GHz, 2 núcleos, Linux 6.18.44-fc-v140 x86_64, Open MPI 4.1.6 y OpenSSL 3.0.13 (Figura 7).

Inciso 1: cifrar

```
$ ./bin/aes_texto -e 123456 -i datos/prueba.txt -o resultados/cifrado.bin
=== AES-128-CBC (cifrar) ===
Clave (parametro):    123456
Clave AES-128:        00000000000000000000000000001e240  [16 B]
IV:                   863b4f0a53909f1037c012e25bea536f  [16 B]
Texto original:       "Esta es una prueba de proyecto 2"  [32 B]
Texto cifrado:        86acd882078af06e092c014221f09194da9fd3f341dd38b37a599ccce94b86eed7b4b85bd4684436ef80262e
eea45204  [48 B]
Archivo cifrado:      resultados/cifrado.bin (IV + cifrado, 64 B)
```

Figura 4. Cifrado: ./bin/aes_texto -e 123456 -i datos/prueba.txt -o resultados/cifrado.bin

Inciso 1: descifrar

```
$ ./bin/aes_texto -d 123456 -i resultados/cifrado.bin -o resultados/recuperado.txt
=== AES-128-CBC (descifrar) ===
Clave (parametro):    123456
Clave AES-128:        00000000000000000000000000001e240  [16 B]
IV (del archivo):     863b4f0a53909f1037c012e25bea536f  [16 B]
Texto cifrado:        86acd882078af06e092c014221f09194da9fd3f341dd38b37a599ccce94b86eed7b4b85bd4684436ef80262e
eea45204  [48 B]
Texto recuperado:     "Esta es una prueba de proyecto 2"  [32 B]
Archivo recuperado:   resultados/recuperado.txt
```

Figura 5. Descifrado: ./bin/aes_texto -d 123456 -i resultados/cifrado.bin -o resultados/recuperado.txt

Inciso 1: clave incorrecta

```
$ ./bin/aes_texto -d 123457 -i resultados/cifrado.bin -o resultados/basura.txt
Error: la clave no descifra el archivo (relleno PKCS#7 invalido): clave incorrecta o archivo danado
=== AES-128-CBC (descifrar) ===
Clave (parametro):    123457
Clave AES-128:        00000000000000000000000000001e241  [16 B]
IV (del archivo):     863b4f0a53909f1037c012e25bea536f  [16 B]
Texto cifrado:        86acd882078af06e092c014221f09194da9fd3f341dd38b37a599ccce94b86eed7b4b85bd4684436ef80262e
eea45204  [48 B]
```

Figura 6. Descifrado con una clave incorrecta: el relleno PKCS#7 no es válido.

4.2 Búsqueda de la clave con 4 procesos

La Tabla 3 resume los tiempos medidos con `mpirun -np 4` para el texto "Esta es una prueba de proyecto 2" y la frase "es una prueba de" (promedio ± desviación estándar muestral de 5 repeticiones cuando la búsqueda termina; cada repetición está en la Tabla 12). Las búsquedas que no terminan en un tiempo razonable se cortaron con `-t` y se reporta la estimación que calcula el programa a partir de la tasa medida. Las capturas están en las Figuras 9 a 17. El equipo tiene 2 núcleos, así que con `-np 4` se usó `--oversubscribe`: dos procesos comparten cada núcleo y la tasa por proceso es menor que la del secuencial.

Tabla 3. Tiempo de búsqueda de cada clave (medido o estimado).

Clave	Estrategia 1, -np 4	Estrategia 2, -np 4	Secuencial
a) 123456	4.94 ± 1.33 ms	19.93 ± 6.49 ms	10.76 ± 3.02 ms
b) 18014398509481984 (2^56/4)	no termina; ≈ 23.1 años (est.)	1.25 ± 0.74 ms	no termina; ≈ 45.0 años (est.)
c) 18014398509481985 (2^56/4 + 1)	no termina; ≈ 22.8 años (est.)	756.80 ± 199.67 µs	no termina; ≈ 44.5 años (est.)

La Tabla 4 indica cuántos intentos debe hacer el proceso que posee cada clave y el tiempo que eso implica con la tasa medida en las corridas acotadas: v ≈ 6.21×10^6 claves/s por proceso con 4 procesos en este equipo (el secuencial alcanza 1.27×10^7 claves/s). La tasa de 2.7×10^6 claves/s del Laboratorio 03 ya no aplica: aquel programa probaba un solo bloque con EVP y este descifra tres bloques con AES-NI.

Tabla 4. Intentos del proceso dueño y tiempo esperado con la tasa medida.

Clave	Proceso dueño	Intentos (estr. 1)	Intentos (estr. 2)	Tiempo esperado (estr. 1 / estr. 2)
a) 123456	0	30 865	123 457	4.97 ms / 19.87 ms
b) 18014398509481984	0 (e1), 1 (e2)	4 503 599 627 370 497	1	≈ 23.0 años / 0.16 µs
c) 18014398509481985	1	4 503 599 627 370 497	2	≈ 23.0 años / 0.32 µs

Las claves b) y c) no pueden medirse con la estrategia 1 ni con el secuencial: se cortaron a los 30 s (3 repeticiones cada una). En ese tiempo los 4 procesos probaron unas 7.40×10^8 (b) y 7.51×10^8 (c) claves sin encontrarla, y el programa estima ≈ 23.1 años para b) y ≈ 22.8 años para c), pues el proceso dueño necesita 2^52 + 1 intentos en ambos casos. Con la estrategia 2 las mismas claves se encontraron en 1.25 ms y 756.80 µs: son la primera y la segunda clave del bloque del proceso 1.

4.3 Por qué cambia el tiempo según la clave

En una búsqueda por fuerza bruta el tiempo no depende del tamaño del espacio sino de cuántas claves se prueban antes de llegar a la correcta, y eso depende de dónde está la clave y del orden en que cada estrategia recorre el rango. Con la estrategia 1, el tiempo es proporcional a la posición de la clave dividida entre p: 123456 se encuentra tras unos 30 mil intentos, mientras que 2^54 exige 2^52. Con la estrategia 2, el tiempo es proporcional al desplazamiento dentro del bloque: 2^54 y 2^54 + 1 son la primera y la segunda clave del bloque del proceso 1 y se encuentran casi al instante, mientras que 123456 está a 123 456 pasos del inicio del bloque del proceso 0.

Las claves 2^54 y 2^54 + 1 difieren en un solo intento con la estrategia 2 y en el proceso que las posee con la estrategia 1, y sus tiempos resultaron del mismo orden dentro de cada estrategia, con diferencias que caen en el ruido de medición (Tabla 3). Para una clave uniformemente aleatoria las dos estrategias tienen el mismo tiempo esperado, N / (2·p·v) en promedio y el mismo peor caso, N / (p·v). La diferencia aparece con claves estructuradas, como números pequeños o límites de bloque.

Las mediciones confirman el análisis. Con la clave a) la estrategia 1 tardó 4.94 ms y la 2 tardó 19.93 ms, una razón de 4.0 frente a la teórica de 123 457 / 30 865 ≈ 4.0. La coincidencia es buena, aunque cada corrida varía bastante (ver las desviaciones de la Tabla 3) porque en tiempos de milisegundos pesan efectos que la cuenta de intentos no incluye: el tiempo reportado es el del último proceso en detenerse, y los demás siguen probando al menos hasta su siguiente revisión, cada 4096 candidatas (≈ 659.16 µs a la tasa medida), y a veces varios intervalos más mientras llega el aviso; además, al haber más procesos que núcleos, el sistema operativo alterna los procesos y la velocidad de cada uno varía entre corridas. Con b) y c) la diferencia entre estrategias es de unas 5.83×10^11 y 9.51×10^11 veces, respectivamente: no por hacer el trabajo más rápido, sino por visitar primero la zona donde está la clave.

4.4 Speedup

El Speedup se calcula como S = T_secuencial / T_paralelo con 4 procesos y la eficiencia como E = S / 4 [7]. Cuando un tiempo no puede medirse se usa su estimación, y el Speedup se marca como estimado (est.). La Tabla 5 compara cada clave y estrategia con el secuencial.

Tabla 5. Speedup y eficiencia de cada prueba con 4 procesos.

Clave	T secuencial	T paralelo	Speedup	Eficiencia	Estrategia
a) 123456	10.76 ms	4.94 ms	2.18	0.55	1
a) 123456	10.76 ms	19.93 ms	0.54	0.13	2
b) 18014398509481984	≈ 45.0 años	≈ 23.1 años	1.94 (est.)	0.49	1
b) 18014398509481984	≈ 45.0 años	1.25 ms	1.13×10^12 (est.)	2.83×10^11	2
c) 18014398509481985	≈ 44.5 años	≈ 22.8 años	1.95 (est.)	0.49	1
c) 18014398509481985	≈ 44.5 años	756.80 µs	1.86×10^12 (est.)	4.64×10^11	2

Los valores de Speedup de la Tabla 5 mezclan dos efectos: el paralelismo y el orden en que cada estrategia recorre el rango. Los valores enormes de la estrategia 2 con b) y c) (del orden de 1.13×10^12) no son aceleración paralela, sino que la clave queda al inicio de un bloque; y el Speedup menor que 1 de la estrategia 2 con a) se debe a que solo el proceso 0 trabaja sobre las claves pequeñas. El Speedup de 2.18 de la estrategia 1 con a) supera levemente el límite de 2 núcleos, pero con tiempos de pocos milisegundos la variación entre corridas es del mismo orden (± 1.33 ms), así que no debe leerse como un Speedup superlineal. Para medir solo el paralelismo se repitió la búsqueda de la última clave de un rango de 2^26 (67 108 863), que en ambas estrategias es la última candidata del último proceso: todos recorren su tramo completo y el trabajo es el mismo. El secuencial tardó 5.330 s (primera fila de la Tabla 6). Con 2 procesos el Speedup fue 2.06–2.08 y con 4 procesos 2.01–2.03 según la estrategia. Con 1 proceso el programa MPI es levemente más rápido que el secuencial (S = 1.05–1.06) y con 2 procesos la eficiencia llega a 1.0: el ciclo de busqueda_mpi y el de busqueda_secuencial no son idénticos y la variación entre corridas es de ± 0.17 s, así que esas diferencias no son significativas. Como el equipo solo tiene 2 núcleos, el Speedup no puede

superar 2: con 4 procesos cada núcleo se reparte entre dos procesos y la tasa por proceso cae a la mitad. En un equipo con 4 o más núcleos se espera un Speedup cercano a 4 con ambas estrategias.

Tabla 6. Escalabilidad con trabajo fijo (rango 2^26, clave 2^26 − 1).

Procesos	Estrategia	Tiempo	Claves/s por proceso	Speedup	Eficiencia
Secuencial	—	5.33 ± 0.17 s	12 601 422	1.00	1.00
1	1	5.07 ± 0.08 s	13 238 015	1.05	1.05
1	2	5.04 ± 0.14 s	13 313 744	1.06	1.06
2	1	2.59 ± 0.08 s	12 944 122	2.06	1.03
2	2	2.56 ± 0.03 s	13 027 542	2.08	1.04
4	1	2.63 ± 0.09 s	6 291 781	2.03	0.51
4	2	2.65 ± 0.15 s	6 275 542	2.01	0.50

La Tabla 7 compara las dos implementaciones de la prueba de candidatas en el programa secuencial con el mismo rango de 2^26 claves (Figuras 18 y 19).

Tabla 7. Tasa de la búsqueda secuencial según la implementación de AES.

Implementación	Tiempo	Claves/s	Aceleración
OpenSSL EVP (-a evp)	14.245 s	4 715 473	1.00
AES-NI (-a aesni)	5.229 s	12 846 680	2.72

5. Discusión, conclusiones y recomendaciones

5.1 Retos y soluciones

- **Rango de 2^56 claves.** Excede los 32 bits; se usa aritmética de 64 bits sin signo y se convierte cada candidata a una clave de 128 bits en big-endian.
- **Frase de búsqueda entre dos bloques.** La frase de 16 bytes cruza el límite de bloque del texto y, en general, su posición es desconocida, así que cada candidata exige descifrar los tres bloques del mensaje. Esto reduce la tasa frente a la prueba de un solo bloque del laboratorio; se compensó con AES-NI.
- **Terminación coordinada.** Se resolvió con un aviso FOUND enviado con MPI_Isend y revisado con MPI_Iprobe cada 4096 candidatas. La primera versión usaba MPI_Send y vaciaba los avisos con un solo MPI_Iprobe; la revisión de código mostró que eso depende del buffer interno de MPI, así que ahora se cuentan y reciben exactamente los avisos pendientes.
- **Pruebas que no terminan.** Las claves b) y c) con la estrategia 1 requieren 2^52 + 1 intentos (≈ 23.1 años en este equipo); se reporta una estimación a partir de corridas acotadas con -t.
- **Mediciones cortas y ruidosas.** Con 123456 el tiempo es de milisegundos y lo afectan la latencia de parada (una o más revisiones de 4096 candidatas por proceso), el reparto de núcleos del sistema operativo y que se reporte el tiempo máximo entre procesos; por eso cada medición se repitió y se reporta promedio y desviación.
- **Rendimiento por candidata.** La API EVP de OpenSSL 3 tiene un costo fijo alto por clave; implementar la expansión de clave y el descifrado con AES-NI aumentó la tasa 2.7 veces sin perder portabilidad (se conserva EVP como respaldo).

5.2 Conclusiones

- El tiempo de una búsqueda por fuerza bruta depende de la posición de la clave en el orden de recorrido, no del tamaño del rango.
- La estrategia 1 reparte el trabajo de forma que todos los procesos colaboran en la misma zona del rango: con la clave a) fue la más rápida (4.94 ms), pero para b) y c) resulta impracticable (≈ 23.1 años).
- La estrategia 2 encontró b) y c) en 1.25 ms y 756.80 µs porque están al inicio de un bloque, pero con a) fue 4.0 veces más lenta que la estrategia 1, porque solo el proceso 0 recorre las claves pequeñas.
- Para una clave uniformemente aleatoria ninguna de las dos estrategias gana: tienen el mismo tiempo esperado y el mismo peor caso. La mejora de la estrategia 2 es válida solo para claves estructuradas.
- Con trabajo fijo, ambas estrategias escalan igual: el Speedup con 2 procesos fue 2.06–2.08 y con 4 procesos 2.01–2.03, limitado por los 2 núcleos del equipo. La optimización con AES-NI aportó otro factor de 2.7 independiente del número de procesos.
- Como aprendizaje, el proyecto mostró que en una búsqueda paralela importa tanto el reparto como la velocidad de cada proceso, que la terminación con MPI debe diseñarse según el estándar y no según el comportamiento observado de una implementación, y que el Speedup solo se interpreta bien si se separa el efecto del orden de recorrido del efecto del paralelismo.

5.3 Recomendaciones

- Si no se conoce dónde está la clave, combinar ambas ideas: repartir el rango en trozos pequeños bajo demanda desde un proceso coordinador, con varios puntos de partida. Como señala [8, sec. 8.1], "This approach can sometimes be an improvement on the assignment of and [sic] equal number of tasks to all processes".[1] El costo es más comunicación con el coordinador, que se puede reducir con trozos grandes.
- Medir siempre con varias repeticiones y con la clave en distintas posiciones del rango (inicio, medio, final, límites de bloque).
- Ejecutar las mediciones en un equipo con al menos 4 núcleos físicos (o en varias máquinas con Open MPI) para observar el Speedup completo; el programa no requiere cambios para eso.
- Recordar que el ejercicio solo es posible porque el rango se restringe; el espacio completo de AES-128 (2^128) sigue siendo inviable por fuerza bruta.

[1] Traducción propia: «este enfoque a veces puede ser una mejora respecto de asignar el mismo número de tareas a todos los procesos» [8].

Anexo 1. Catálogo de funciones y librerías

Rutinas compartidas (include/aes_texto.h, src/common/aes_texto.c), usadas por los tres programas:

Tabla 8. Rutinas compartidas del proyecto.

Función	Entradas	Salida	Propósito
reportar_error	mensaje	—	Imprime el error y la cola de errores de OpenSSL (no termina el programa).
reloj	—	segundos (double)	Reloj monótono (CLOCK_MONOTONIC) para el secuencial.
parse_u64_seguro	texto, *valor	0 / -1	Convierte decimal o 0x… a uint64_t; rechaza signos, espacios y desbordamientos.
parse_clave_hex, valor_hex	texto, clave[16]	0 / -1	Lee una clave o IV de 32 dígitos hexadecimales (valor_hex convierte un dígito).
imprimir_hex, imprimir_texto	datos, n	—	Muestran bytes en hex o como texto con escapes (\n, \xNN).
imprimir_duracion	segundos	—	Imprime una duración y su equivalente en horas, días o años.
leer_archivo	ruta, máximo, *buffer	0 / -1	Lee un archivo completo por bloques con límite de tamaño; valida apertura, lectura y archivo vacío.
escribir_archivo	ruta, datos, n	0 / -1	Escribe un archivo y verifica fwrite y fclose.
buffer_liberar	*buffer	—	Libera un buffer_t y lo deja vacío (seguro llamarla dos veces).
clave_desde_u64	k, clave[16]	clave	Convierte el entero k en 16 bytes big-endian con ceros a la izquierda.
cifrar_texto	clave, IV, texto, n, salida	longitud / -1	AES-128-CBC con relleno PKCS#7 (EVP).
descifrar_texto	clave, IV, cifrado, n, salida	longitud / -1	Descifra y valida el relleno; -1 si la clave es incorrecta.
contiene_frase	texto, n, frase, m	1 / 0	Busca la frase en cualquier posición (memchr + memcmp).
aesni_disponible	—	1 / 0	Detecta en tiempo de ejecución si la CPU tiene AES-NI y SSE4.1.
expandir_paso, subclaves_descifrado	clave[16], dk[11]	subclaves	Expansión de clave de AES-128 con AES-NI y subclaves de descifrado (InvMixColumns).
descifrar_cbc_aesni	clave, IV, cifrado, n, salida	—	Descifra CBC bloque a bloque con aesdec/aesdeclast: P_i = D(C_i) xor C_(i-1).
nombre_motor, nombre_estrategia	motor o estrategia	texto	Nombre legible para los reportes.
buscador_iniciar	*b, IV, cifrado, n, frase, m, motor	0 / -1	Prepara la prueba de candidatas: elige EVP o AES-NI, crea el contexto y el buffer de trabajo.
probar_clave	*b, k	1 / 0 / -1	Descifra el mensaje con la clave k, busca la frase y, si aparece, valida el relleno.
relleno_valido	texto, n	1 / 0	Revisa que el último bloque termine con un relleno PKCS#7 válido.

Función	Entradas	Salida	Propósito
buscador_liberar	*b	—	Libera el contexto EVP y el buffer.
calcular_tramo	estrategia, N, p, rank	tramo_t	Inicio, paso y cantidad de claves del proceso (intercalado o bloques con residuo).
ubicar_clave	estrategia, N, p, k	rank, intentos	Qué proceso tiene la clave k y cuántos intentos le toma; para estimar tiempos.
parse_opciones	argc, argv, …	0 / -1	Valida las opciones -i -k -c -f -n -t -e -a de los programas de búsqueda.
preparar_mensaje	opciones, *mensaje	0 / -1	Cifra -i con -k (IV con RAND_bytes) o lee -c (IV + cifrado).
mensaje_liberar	*mensaje	—	Libera el mensaje cifrado.

Funciones de cada programa (src/proyecto02):

Tabla 9. Funciones de los programas del proyecto.

Función	Entradas	Salida	Propósito
uso (en cada programa)	nombre del programa	—	Muestra la forma de uso cuando faltan o sobran argumentos.
aes_texto: leer_clave	texto, clave[16]	0 / -1	Acepta la clave como entero (123456, 0x1e240) o hex:<32 dígitos>.
aes_texto: mostrar_hex, mostrar_texto	etiqueta, datos, n	—	Imprimen un campo en hex (máx. 96 B) o como texto, con su tamaño.
aes_texto: cifrar_archivo	clave, IV, texto, salida	0 / -1	Cifra y guarda IV || cifrado; muestra texto original y cifrado.
aes_texto: descifrar_archivo	clave, archivo, salida	0 / -1	Separa el IV, descifra, valida el relleno y guarda el texto recuperado.
aes_texto: main	-e/-d/-r CLAVE -i -o [-v IV]	código de salida	Modos cifrar, descifrar e ida y vuelta (verifica que el texto recuperado sea idéntico).
busqueda_secuencial: main	-i -k | -c, -f -n -t -a	código de salida	Recorre k = 0, 1, 2, … por lotes de 4096 revisando el límite de tiempo; reporta clave, texto, tiempo y tasa.
busqueda_mpi: difundir_mensaje	opciones, rank, *mensaje	0 / -1	El proceso 0 prepara el mensaje y lo difunde con MPI_Bcast; si algún proceso no puede reservar memoria, todos devuelven -1 juntos (MPI_Allreduce con MPI_LAND).
busqueda_mpi: buscar_en_tramo	*b, tramo, límite, inicio, *r	— (llena *r)	Recorre el tramo del proceso; cada 4096 candidatas revisa avisos y el límite de tiempo.
busqueda_mpi: avisar_encontrada	*k, rank, size, reqs	n° de envíos	Envía la clave a los demás procesos con MPI_Isend (etiqueta FOUND).
busqueda_mpi: hay_aviso_found	*k	1 / 0	MPI_Iprobe sin bloquear; si hay aviso lo recibe con MPI_Recv.
busqueda_mpi: recibir_avisos_pendientes	resultado local	—	Cuenta los avisos enviados (MPI_Allreduce) y recibe los que faltan antes de MPI_Finalize.
busqueda_mpi: imprimir_encabezado, imprimir_clave	opciones, mensaje, …	—	Reporte del proceso 0: estrategia, motor, clave, proceso dueño, intentos y texto.
busqueda_mpi: main	-i -k | -c, -e -f -n -t -a	código de salida	Inicializa MPI, reparte, busca, reduce resultados (clave, tiempo máximo, claves probadas) y reporta.

Librerías:

Tabla 10. Librerías y funciones externas usadas.

Librería	Funciones usadas y propósito
OpenSSL (EVP)	EVP_CIPHER_CTX_new/free, EVP_EncryptInit_ex, EVP_DecryptInit_ex, EVP_EncryptUpdate, EVP_DecryptUpdate, EVP_EncryptFinal_ex, EVP_DecryptFinal_ex, EVP_CIPHER_CTX_set_padding, EVP_aes_128_cbc: cifrado y descifrado AES-128-CBC.
OpenSSL (rand, err)	RAND_bytes: IV aleatorio. ERR_print_errors_fp, ERR_clear_error: errores de OpenSSL.
AES-NI (immintrin.h)	_mm_aeskeygenassist_si128, _mm_aesimc_si128, _mm_aesdec_si128, _mm_aesdeclast_si128, _mm_shuffle_epi32, _mm_slli_si128, _mm_loadu/storeu/xor_si128; __builtin_cpu_init y __builtin_cpu_supports para detectar la CPU.
Open MPI	MPI_Init, MPI_Comm_rank, MPI_Comm_size, MPI_Bcast, MPI_Barrier, MPI_Wtime, MPI_Isend, MPI_Iprobe, MPI_Recv, MPI_Waitall, MPI_Allreduce, MPI_Reduce, MPI_Abort, MPI_Finalize.
Biblioteca estándar de C	stdio.h, stdlib.h, string.h, stdint.h, inttypes.h, ctype.h, errno.h, math.h, limits.h, time.h (clock_gettime) y unistd.h (getopt).

Anexo 2. Bitácora de pruebas y speedups

Todas las pruebas se generaron con `scripts/bitacora.sh` (`make bitacora`), que guarda la salida exacta de cada comando en `resultados/salidas/`, los tiempos en `resultados/tiempos.csv` y las capturas de este anexo. Equipo: Intel(R) Xeon(R) Processor @ 2.10GHz, 2 núcleos, Linux 6.18.44-fc-v140 x86_64, Open MPI 4.1.6, OpenSSL 3.0.13. La Tabla 11 resume cada prueba con la figura que la evidencia.

Tabla 11. Bitácora de pruebas (tiempos en segundos: promedio ± desviación).

No.	Prueba y comando	Resultado	Tiempo (s)	Captura
1	Compilación: `make`	Sin advertencias	—	Figura 8
2	Cifrar: `./bin/aes_texto -e 123456 -i datos/prueba.txt -o resultados/cifrado.bin`	Cifrado de 48 B	—	Figura 4
3	Descifrar: `./bin/aes_texto -d 123456 -i resultados/cifrado.bin -o resultados/recuperado.txt`	Texto idéntico	—	Figura 5
4	Búsqueda a), estrategia 1: `mpirun --oversubscribe -np 4 ./bin/busqueda_mpi -i datos/prueba.txt -k 123456 -e 1`	Encontrada	0.0049 ± 0.0013	Figura 9
5	Búsqueda a), estrategia 2: `mpirun --oversubscribe -np 4 ./bin/busqueda_mpi -i datos/prueba.txt -k 123456 -e 2`	Encontrada	0.0199 ± 0.0065	Figura 10
6	Búsqueda b), estrategia 1: `mpirun --oversubscribe -np 4 ./bin/busqueda_mpi -i datos/prueba.txt -k 18014398509481984 -e 1 -t 30`	Cortada a 30.0 s	30.001 medido; ≈ 23.1 años est.	Figura 11
7	Búsqueda b), estrategia 2: `mpirun --oversubscribe -np 4 ./bin/busqueda_mpi -i datos/prueba.txt -k 18014398509481984 -e 2`	Encontrada	0.0013 ± 0.0007	Figura 12
8	Búsqueda c), estrategia 1: `mpirun --oversubscribe -np 4 ./bin/busqueda_mpi -i datos/prueba.txt -k 18014398509481985 -e 1 -t 30`	Cortada a 30.0 s	30.000 medido; ≈ 22.8 años est.	Figura 13
9	Búsqueda c), estrategia 2: `mpirun --oversubscribe -np 4 ./bin/busqueda_mpi -i datos/prueba.txt -k 18014398509481985 -e 2`	Encontrada	0.0008 ± 0.0002	Figura 14
10	Secuencial a): `./bin/busqueda_secuencial -i datos/prueba.txt -k 123456`	Encontrada	0.0108 ± 0.0030	Figura 15
11	Secuencial b): `./bin/busqueda_secuencial -i datos/prueba.txt -k 18014398509481984 -t 30`	Cortada a 30.0 s	30.000 medido; ≈ 45.0 años est.	Figura 16

No.	Prueba y comando	Resultado	Tiempo (s)	Captura
12	Secuencial c): `./bin/busqueda_secuencial -i datos/prueba.txt -k 18014398509481985 -t 30`	Cortada a 30.0 s	30.000 medido; ≈ 44.5 años est.	Figura 17
13	Escalabilidad (Tabla 6): `./bin/busqueda_secuencial -i datos/prueba.txt -k 67108863 -n 26` y las mismas con mpirun -np 1, 2, 4 y -e 1|2	Encontrada	5.330 ± 0.165 (sec.)	resultados/ salidas/
14	Motor evp: `./bin/busqueda_secuencial -i datos/prueba.txt -k 67108863 -n 26 -a evp`	Encontrada	14.245 ± 0.476	Figura 18
15	Motor aesni: `./bin/busqueda_secuencial -i datos/prueba.txt -k 67108863 -n 26 -a aesni`	Encontrada	5.229 ± 0.182	Figura 19

Cálculo del Speedup: S = T_secuencial / T_paralelo y E = S / p, con p = 4 (Tabla 5). Por ejemplo, para la clave a) con la estrategia 1: S = 10.76 ms / 4.94 ms = 2.18 y E = 0.55. La Tabla 12 muestra cada repetición.

Tabla 12. Tiempo de cada repetición (s).

Clave	Programa	P	Repeticiones (s)	Promedio
a)	secuencial	1	0.0102; 0.0092; 0.0161; 0.0090; 0.0093	0.0108 ± 0.0030
a)	MPI e1	4	0.0045; 0.0043; 0.0040; 0.0046; 0.0073	0.0049 ± 0.0013
a)	MPI e2	4	0.0208; 0.0099; 0.0200; 0.0209; 0.0281	0.0199 ± 0.0065
b)	secuencial	1	30.0001; 30.0001; 30.0001	≈ 45.0 años (est.)
b)	MPI e1	4	30.0006; 30.0004; 30.0008	≈ 23.1 años (est.)
b)	MPI e2	4	0.0015; 0.0007; 0.0010; 0.0024; 0.0007	0.0013 ± 0.0007
c)	secuencial	1	30.0001; 30.0002; 30.0003	≈ 44.5 años (est.)
c)	MPI e1	4	30.0006; 30.0007; 30.0002	≈ 22.8 años (est.)
c)	MPI e2	4	0.0006; 0.0010; 0.0006; 0.0009; 0.0006	0.0008 ± 0.0002



Figura 7. Equipo y versiones usadas en las pruebas.

```
Compilacion (make)

$ make
cc -Iinclude  -std=c11 -O2 -Wall -Wextra src/lab03/busqueda_clave_aes_secuencial.c -o bin/busqueda_clave_aes_s
ecuencial  -lcrypto
cc -Iinclude  -std=c11 -O2 -Wall -Wextra -c src/lab03/busqueda_clave_aes_secuencial_mejorado.c -o build/lab03/
busqueda_clave_aes_secuencial_mejorado.o
cc -Iinclude  -std=c11 -O2 -Wall -Wextra -c src/common/aes_busqueda.c -o build/common/aes_busqueda.o
cc build/lab03/busqueda_clave_aes_secuencial_mejorado.o build/common/aes_busqueda.o -o bin/busqueda_clave_aes_
secuencial_mejorado  -lcrypto
cc -Iinclude  -std=c11 -O2 -Wall -Wextra -c src/proyecto02/aes_texto.c -o build/proyecto02/aes_texto.o
cc -Iinclude  -std=c11 -O2 -Wall -Wextra -c src/common/aes_texto.c -o build/common/aes_texto.o
cc build/proyecto02/aes_texto.o build/common/aes_texto.o -o bin/aes_texto  -lcrypto
mpicc -Iinclude  -std=c11 -O2 -Wall -Wextra -c src/proyecto02/busqueda_mpi.c -o build/proyecto02/busqueda_mpi.
o
mpicc build/proyecto02/busqueda_mpi.o build/common/aes_texto.o -o bin/busqueda_mpi  -lcrypto
cc -Iinclude  -std=c11 -O2 -Wall -Wextra -c src/proyecto02/busqueda_secuencial.c -o build/proyecto02/busqueda_
secuencial.o
cc build/proyecto02/busqueda_secuencial.o build/common/aes_texto.o -o bin/busqueda_secuencial  -lcrypto
```

Figura 8. Compilación con make (mpicc para busqueda_mpi).

```
Inciso 2a) estrategia 1, -np 4

$ mpirun --oversubscribe -np 4 ./bin/busqueda_mpi -i datos/prueba.txt -k 123456 -e 1
=== Busqueda AES-128 por fuerza bruta (MPI, 4 procesos) ===
Estrategia:           1 - reparto intercalado (base)
Rango:                [0, 2^56) = 72057594037927936 candidatas
Frase de busqueda:    "es una prueba de"
Motor AES:            AES-NI (instrucciones x86)
Resultado:            CLAVE ENCONTRADA
Clave (decimal):      123456
Clave AES-128:        000000000000000000000000001e240
Hallada por proceso:  0 (tras 30865 intentos en su tramo, 0.003832 s)
Texto descifrado:     "Esta es una prueba de proyecto 2"
Claves probadas:      125073 (suma de los 4 procesos)
Tiempo de busqueda:   0.004543 s
Tasa agregada:        27529006 claves/s (6882251 por proceso)
```

Figura 9. Búsqueda de la clave 123456 con la estrategia 1 y 4 procesos.

```
Inciso 2a) estrategia 2, -np 4

$ mpirun --oversubscribe -np 4 ./bin/busqueda_mpi -i datos/prueba.txt -k 123456 -e 2
=== Busqueda AES-128 por fuerza bruta (MPI, 4 procesos) ===
Estrategia:           2 - bloques contiguos (alternativa)
Rango:                [0, 2^56) = 72057594037927936 candidatas
Frase de busqueda:    "es una prueba de"
Motor AES:            AES-NI (instrucciones x86)
Resultado:            CLAVE ENCONTRADA
Clave (decimal):      123456
Clave AES-128:        0000000000000000000000001e240
Hallada por proceso:  0 (tras 123457 intentos en su tramo, 0.019713 s)
Texto descifrado:     "Esta es una prueba de proyecto 2"
Claves probadas:      512577 (suma de los 4 procesos)
Tiempo de busqueda:   0.020758 s
Tasa agregada:        24693471 claves/s (6173368 por proceso)
```

Figura 10. Búsqueda de la clave 123456 con la estrategia 2 y 4 procesos.

```
Inciso 2b) estrategia 1, -np 4

$ mpirun --oversubscribe -np 4 ./bin/busqueda_mpi -i datos/prueba.txt -k 18014398509481984 -e 1 -t 30
=== Busqueda AES-128 por fuerza bruta (MPI, 4 procesos) ===
Estrategia:           1 - reparto intercalado (base)
Rango:                [0, 2^56) = 72057594037927936 candidatas
Frase de busqueda:    "es una prueba de"
Motor AES:            AES-NI (instrucciones x86)
Limite de tiempo:     30.0 s
Resultado:            NO ENCONTRADA (limite de 30.0 s alcanzado)
Claves probadas:      737677312 (suma de los 4 procesos)
Tiempo de busqueda:   30.000632 s
Tasa agregada:        24588725 claves/s (6147181 por proceso)
Estimacion para la clave 18014398509481984 (estrategia 1):
  proceso 0, 4503599627370497 intentos -> 732628398.448202 s (~23.2 anios)
```

Figura 11. Búsqueda de la clave 18014398509481984 (2^56/4) con la estrategia 1 y 4 procesos.

```
Inciso 2b) estrategia 2, -np 4

$ mpirun --oversubscribe -np 4 ./bin/busqueda_mpi -i datos/prueba.txt -k 18014398509481984 -e 2
=== Busqueda AES-128 por fuerza bruta (MPI, 4 procesos) ===
Estrategia:           2 - bloques contiguos (alternativa)
Rango:                [0, 2^56) = 72057594037927936 candidatas
Frase de busqueda:    "es una prueba de"
Motor AES:            AES-NI (instrucciones x86)
Resultado:            CLAVE ENCONTRADA
Clave (decimal):      18014398509481984
Clave AES-128:        00000000000000000040000000000000
Hallada por proceso:  1 (tras 1 intentos en su tramo, 0.000001 s)
Texto descifrado:     "Esta es una prueba de proyecto 2"
Claves probadas:      45057 (suma de los 4 procesos)
Tiempo de busqueda:   0.001486 s
Tasa agregada:        30319915 claves/s (7579979 por proceso)
```

Figura 12. Búsqueda de la clave 18014398509481984 (2^56/4) con la estrategia 2 y 4 procesos.

```
Inciso 2c) estrategia 1, -np 4

$ mpirun --oversubscribe -np 4 ./bin/busqueda_mpi -i datos/prueba.txt -k 18014398509481985 -e 1 -t 30
=== Busqueda AES-128 por fuerza bruta (MPI, 4 procesos) ===
Estrategia:           1 - reparto intercalado (base)
Rango:                [0, 2^56) = 72057594037927936 candidatas
Frase de busqueda:    "es una prueba de"
Motor AES:            AES-NI (instrucciones x86)
Limite de tiempo:     30.0 s
Resultado:            NO ENCONTRADA (limite de 30.0 s alcanzado)
Claves probadas:      758804480 (suma de los 4 procesos)
Tiempo de busqueda:   30.000587 s
Tasa agregada:        25292988 claves/s (6323247 por proceso)
Estimacion para la clave 18014398509481985 (estrategia 1):
  proceso 1, 4503599627370497 intentos -> 712228972.766276 s (~22.6 anios)
```

Figura 13. Búsqueda de la clave 18014398509481985 (2^56/4 + 1) con la estrategia 1 y 4 procesos.

```
Inciso 2c) estrategia 2, -np 4

$ mpirun --oversubscribe -np 4 ./bin/busqueda_mpi -i datos/prueba.txt -k 18014398509481985 -e 2
=== Busqueda AES-128 por fuerza bruta (MPI, 4 procesos) ===
Estrategia:           2 - bloques contiguos (alternativa)
Rango:                [0, 2^56) = 72057594037927936 candidatas
Frase de busqueda:    "es una prueba de"
Motor AES:            AES-NI (instrucciones x86)
Resultado:            CLAVE ENCONTRADA
Clave (decimal):      18014398509481985
Clave AES-128:        00000000000000000040000000000001
Hallada por proceso:  1 (tras 2 intentos en su tramo, 0.000001 s)
Texto descifrado:     "Esta es una prueba de proyecto 2"
Claves probadas:      16386 (suma de los 4 procesos)
Tiempo de busqueda:   0.000636 s
Tasa agregada:        25756092 claves/s (6439023 por proceso)
```

Figura 14. Búsqueda de la clave 18014398509481985 (2^56/4 + 1) con la estrategia 2 y 4 procesos.

```
Secuencial, clave a)

$ ./bin/busqueda_secuencial -i datos/prueba.txt -k 123456
=== Busqueda AES-128 por fuerza bruta (secuencial) ===
Rango:                [0, 2^56) = 72057594037927936 candidatas
Frase de busqueda:    "es una prueba de"
Motor AES:            AES-NI (instrucciones x86)
Texto cifrado:        572969cd939bdce2baa833acc257c88bc5b93c13f3f658c0c31334fe55523c41a66859e0a2a1154d55db85f0
08675fc5  [48 B]
Resultado:            CLAVE ENCONTRADA
Clave (decimal):      123456
Clave AES-128:        0000000000000000000000000001e240
Texto descifrado:     "Esta es una prueba de proyecto 2"
Claves probadas:      123457
Tiempo de busqueda:   0.010194 s
Tasa:                 12110643 claves/s
```

Figura 15. Búsqueda secuencial de la clave 123456.

Secuencial, clave b)

```
$ ./bin/busqueda_secuencial -i datos/prueba.txt -k 18014398509481984 -t 30
=== Busqueda AES-128 por fuerza bruta (secuencial) ===
Rango:               [0, 2^56) = 72057594037927936 candidatas
Frase de busqueda:   "es una prueba de"
Motor AES:           AES-NI (instrucciones x86)
Texto cifrado:       4c88cd6f304391c9ff37ca561e0fcc5f09961a8deaff2f712750804c9c9a5b752156ce8c1b8d0ceb595a0378
1211ec09  [48 B]
Limite de tiempo:    30.0 s
Resultado:           NO ENCONTRADA (se alcanzo el limite de 30.0 s)
Claves probadas:     370786304
Tiempo de busqueda:  30.000091 s
Tasa:                12359506 claves/s
Estimacion para la clave 18014398509481984: 18014398509481985 intentos -> 1457533857.563011 s (~46.2 anios)
```

Figura 16. Búsqueda secuencial de la clave 18014398509481984 (2^56/4).

Secuencial, clave c)

```
$ ./bin/busqueda_secuencial -i datos/prueba.txt -k 18014398509481985 -t 30
=== Busqueda AES-128 por fuerza bruta (secuencial) ===
Rango:               [0, 2^56) = 72057594037927936 candidatas
Frase de busqueda:   "es una prueba de"
Motor AES:           AES-NI (instrucciones x86)
Texto cifrado:       bb15b2900db374452819645d9bc85c7dd4854c03c94aca9c041a9d8bdd80ee684fd1f4098820d7d5c4308fcd
a454b96f  [48 B]
Limite de tiempo:    30.0 s
Resultado:           NO ENCONTRADA (se alcanzo el limite de 30.0 s)
Claves probadas:     384831488
Tiempo de busqueda:  30.000083 s
Tasa:                12827681 claves/s
Estimacion para la clave 18014398509481985: 18014398509481986 intentos -> 1404337913.607314 s (~44.5 anios)
```

Figura 17. Búsqueda secuencial de la clave 18014398509481985 (2^56/4 + 1).

Motor AES: evp

```
$ ./bin/busqueda_secuencial -i datos/prueba.txt -k 67108863 -n 26 -a evp
=== Busqueda AES-128 por fuerza bruta (secuencial) ===
Rango:               [0, 2^26) = 67108864 candidatas
Frase de busqueda:   "es una prueba de"
Motor AES:           OpenSSL EVP
Texto cifrado:       4fab307ab970d98a2beabf5d38a59cf727b845894ea1baea86fdcb5fa887eba86819fd58683331f760044abd
eff60b70  [48 B]
Resultado:           CLAVE ENCONTRADA
Clave (decimal):     67108863
Clave AES-128:       00000000000000000000000003ffffff
Texto descifrado:    "Esta es una prueba de proyecto 2"
Claves probadas:     67108864
Tiempo de busqueda:  14.221485 s
Tasa:                4718837 claves/s
```

Figura 18. Búsqueda secuencial con OpenSSL EVP (-a evp).

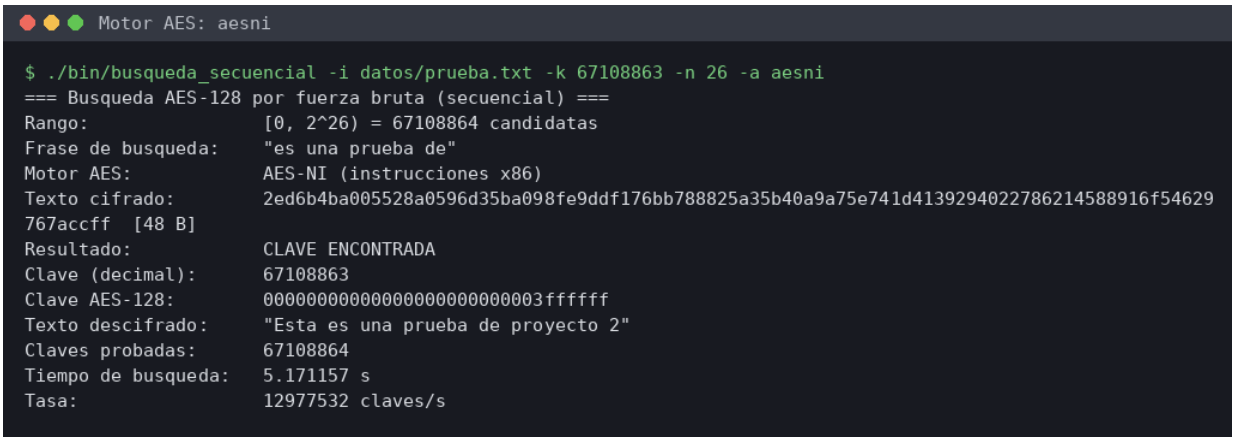


Figura 19. Búsqueda secuencial con AES-NI (-a aesni).

Referencias

[1] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS PUB 197, nov. 2001 (act. 2023). doi: 10.6028/NIST.FIPS.197-upd1.

[2] J. Daemen y V. Rijmen, The Design of Rijndael: AES – The Advanced Encryption Standard. Berlín, Alemania: Springer, 2002. doi: 10.1007/978-3-662-04722-4.

[3] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Methods and Techniques," NIST SP 800-38A, dic. 2001. doi: 10.6028/NIST.SP.800-38A.

[4] OpenSSL Project, "EVP_EncryptInit," OpenSSL 3.0 Manual. [En línea]. Disponible: https://docs.openssl.org/3.0/man3/EVP_EncryptInit/

[5] Message Passing Interface Forum, MPI: A Message-Passing Interface Standard, Version 4.1, nov. 2023. [En línea]. Disponible: https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf

[6] Open MPI Project, "MPI_Iprobe(3)" y "MPI_Isend(3)," Open MPI Documentation. [En línea]. Disponible: https://docs.open-mpi.org/en/main/man-openmpi/man3/MPI_Iprobe.3.html

[7] P. S. Pacheco, An Introduction to Parallel Programming. Burlington, MA, EE. UU.: Morgan Kaufmann, 2011.

[8] Macalester College (CSinParallel), "8.1 Dynamic Load Balancing," en Distributed Computing Fundamentals Using Python. [En línea]. Disponible: https://selkie.macalester.edu/DistributedPython/08DynamicLoadBalance/17dynamicLoad.html